

Encryption of payload for Interoperability

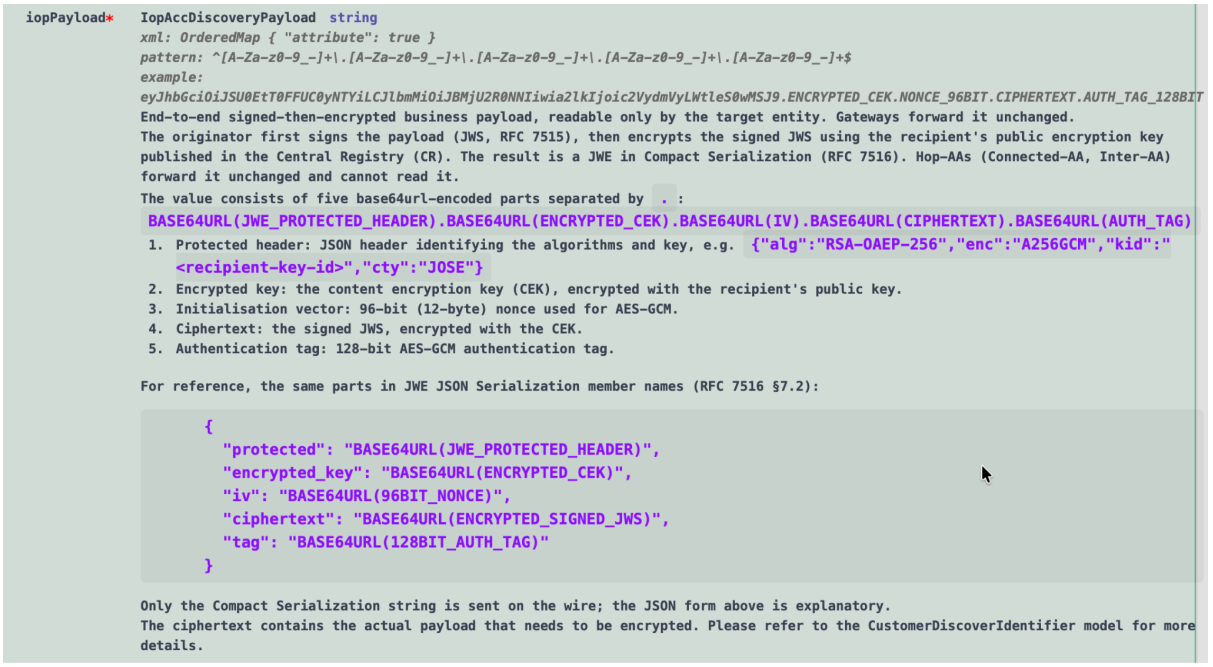
v1.0

Date	Author	Reviewer	Remarks
1.0	Jithesh Kumar		Initial document for encryption

Purpose **2**
Illustrative flow considered **2**
The Default payload **3**
The Envelope payload **4**
The five parts of iopPayload **5**
Protected header parameters 5
Generation processing order (sender) **6**
Recipient processing order **6**
Note **6**

Purpose

This document explains what is the encryption involved in interoperability APIs



iopPayload* IopAccDiscoveryPayload string
xml: OrderedMap { "attribute": true }
pattern: ^[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+$
example:
eyJhbGciOiJSU0EtT0FFUC0yNTYiLCJlbmMiOiJBMjU2R0NNIiwia2lkIjoic2VydmVyLWtleS0wMSJ9.ENCRYPTED_CEK.NONCE_96BIT.CIPHERTEXT.AUTH_TAG_128BIT
End-to-end signed-then-encrypted business payload, readable only by the target entity. Gateways forward it unchanged.
The originator first signs the payload (JWS, RFC 7515), then encrypts the signed JWS using the recipient's public encryption key published in the Central Registry (CR). The result is a JWE in Compact Serialization (RFC 7516). Hop-AAs (Connected-AA, Inter-AA) forward it unchanged and cannot read it.
The value consists of five base64url-encoded parts separated by . :
BASE64URL(JWE_PROTECTED_HEADER).BASE64URL(ENCRYPTED_CEK).BASE64URL(IV).BASE64URL(CIPHERTEXT).BASE64URL(AUTH_TAG)
1. Protected header: JSON header identifying the algorithms and key, e.g. {"alg":"RSA-OAEP-256","enc":"A256GCM","kid":"<recipient-key-id>","cty":"JOSE"}
2. Encrypted key: the content encryption key (CEK), encrypted with the recipient's public key.
3. Initialisation vector: 96-bit (12-byte) nonce used for AES-GCM.
4. Ciphertext: the signed JWS, encrypted with the CEK.
5. Authentication tag: 128-bit AES-GCM authentication tag.

For reference, the same parts in JWE JSON Serialization member names (RFC 7516 §7.2):

```
{
  "protected": "BASE64URL(JWE_PROTECTED_HEADER)",
  "encrypted_key": "BASE64URL(ENCRYPTED_CEK)",
  "iv": "BASE64URL(96BIT_NONCE)",
  "ciphertext": "BASE64URL(ENCRYPTED_SIGNED_JWS)",
  "tag": "BASE64URL(128BIT_AUTH_TAG)"
}
```

Only the Compact Serialization string is sent on the wire; the JSON form above is explanatory.
The ciphertext contains the actual payload that needs to be encrypted. Please refer to the CustomerDiscoverIdentifier model for more details.

iopPayload carries the API's business payload end-to-end so that only the intended recipient can read it. The originator signs the payload (JWS) and then encrypts it using the recipient's public encryption key published in the Central Registry (CR). The result is a JWE in Compact Serialization (RFC 7516). Hops (Connected-AA, Inter-AA) forward it unchanged and cannot read it.

Illustrative flow considered

In this document, we'll look at a scenario where an FIU wants to make an /FI/request call to an AA that is not connected to it.

FIU: fiu-4 (the fiu where the customer has approached)

AA: aa-3 (the aa where the customer's handle and the consent reside)

Connectivity:

- fiu-4 has no connectivity to aa-3
- fiu-4 is contracted with aa-4

Available connectivity between fiu-4 and aa-3:

FIU(fiu-4) ⟶ Connected-AA(aa-4) ⟶ Customer-AA(aa-3)

The Default payload

The default payload that the FIU would generate for an /FI/request call to AA as per the existing ReBIT specifications is as below:

```
Headers:
client_api_key = < Sahamati provided token for fiu-4 >
x-jws-signature = < fiu-4 generated signature of the below json >

Body:
{
  "ver": "2.0.0",
  "timestamp": "2026-10-07T07:35:00.000Z",
  "txnid": "e8cc6822-d4bb-4eb1-9e1b-4996fbff8acb",
  "FIDataRange": {
    "from": "2023-07-06T11:39:57.153Z",
    "to": "2019-12-06T11:39:57.153Z"
  },
  "Consent": {
    "id": "654024c8-29c8-11e8-8868-0289437bf331",
    "digitalSignature": "NdjwMjLortTb10dxcJezkvdOxPVvEdZvIqwq.....Ku78F69Z9Ig"
  },
  "KeyMaterial": {
    "cryptoAlg": "ECDH",
    "curve": "Curve25519",
    "params": "cipher=AES/GCM/NoPadding;KeyPairGenerator=ECDH",
    "DHPublicKey": {
      "expiry": "2023-07-06T11:39:57.153Z",
      "Parameters": "string",
      "KeyValue": "string"
    },
    "Nonce": "29512b70-ca84-46b5-9471-63765599cf15"
  }
}
```

The Envelope payload

Since fiu-4 doesn't have direct connectivity to aa-3, it decides to use aa-4 as a hop to get to aa-3, using the /iop/FI/request instead.

So, it encrypts the above json payload into "iopPayload" and generates a new envelope (or wrapper) request for the /iop/FI/request API, as below:

```
Headers:
client_api_key = < Sahamati provided token for fiu-4 >
x-jws-signature = < fiu-4 generated signature of the below wrapper json >

Body:
{
  "ver": "1.0.0",
  "timestamp": "2026-10-07T07:35:00.000Z",
  "txnid": "4a4adbbe-29ae-11e8-a8d7-0289437bf331",
  "correlationId": "550e8400-e29b-41d4-a716-446655440000",
  "Target": {
    "type": "AA",
    "id": "aa-3"
  },
  "iopPayload": "<PROTECTED_HEADER>.<ENCRYPTED_KEY>.<IV>.<CIPHERTEXT>.<TAG>"
}
```

iopPayload field in the envelope is the signed and encrypted original request as per /FI/request call.

On the Connected-AA ⟶ Customer-AA hop, **Target** is replaced by **Initiator** (type, id of the originator) and iopPayload is passed through unchanged.

```
Headers:
conn_api_key = < Sahamati provided token for aa-4 >
x-jws-signature = < aa-4 generated signature of the below wrapper json >

Body:
{
  "ver": "1.0.0",
  "timestamp": "2026-10-07T07:35:00.010Z",
  "txnid": "5a4adbbe-29ae-11e8-a8d7-0289437bf331",
  "correlationId": "550e8400-e29b-41d4-a716-446655440000",
  "Initiator": {
    "type": "FIU",
    "id": "fiu-4"
  },
  "iopPayload": "<PROTECTED_HEADER>.<ENCRYPTED_KEY>.<IV>.<CIPHERTEXT>.<TAG>"
}
```

Note: The timestamp changes, the txnid changes. The connected-AA must retain the correlationId when forwarding the iopPayload. The iopPayload is forwarded as it is, i.e., it is still the signed and then encrypted original payload corresponding to /FI/request API.

Envelope payload for Error

Let's assume, for the FI/request call, the customer-AA returns an error, say Idempotency error, 409-Conflict.

The business payload in the response would be the following

```
Headers:
aa_api_key = < Sahamati provided token for aa-3 >
x-jws-signature = < aa-3 generated signature of the below wrapper json >

Body:
{
  "ver": "2.0.0",
  "txnid": "0b811819-9044-4856-b0ee-8c88035f8858'",
  "timestamp": "2023-06-26T11:33:34.509Z",
  "errorCode": "IdempotencyError",
  "errorMsg": "Error code specific error message"
}
```

However, since its an /iop/FI/request call, the above json payload would be signed and encrypted into iopError, as below

```
Headers:
cust_api_key = < Sahamati provided token for aa-3 >
x-jws-signature = < aa-3 generated signature of the below wrapper json >

Body:
{
  "ver": "2.0.0",
  "txnid": "0b811819-9044-4856-b0ee-8c88035f8858",
  "timestamp": "2023-06-26T11:33:34.509Z",
  "correlationId": "550e8400-e29b-41d4-a716-446655440000",
  "iopError":
"eyJhbGciOiJSU0EtT0FFUC0yNTYiLCJlbmMiOiJBMjU2R0NNIiwia2lkIjoic2VydmVyLWtleS0wMSJ
9.ENCRYPTED_CEK.NONCE_96BIT.CIPHERTEXT.AUTH_TAG_128BIT"
}
```

The five parts of iopPayload

The parts appear in the order below, separated by "`.`", each base64url-encoded without padding (RFC 7516 7.1).

#	Part	Content	How it is produced	Size
1	Protected header	JSON object describing how the payload was encrypted	Written by the sender and base64url-encoded. The encoded value is also the AES-GCM additional authenticated data, so any change to it makes decryption fail.	Variable (≈100–150 characters)
2	Encrypted key	The content encryption key (CEK), wrapped with the recipient's public key.	A fresh random 256-bit CEK is generated for every message and wrapped with the recipient's encryption key from the CR, identified by kid.	256 bytes (RSA-2048); 512 bytes (RSA-4096)
3	Initialization vector (IV)	The AES-GCM nonce.	Fresh random value for every message; never reused with the same CEK.	12 bytes (96 bits)
4	Ciphertext	The encrypted plaintext. The plaintext is the originator's signed message (compact JWS).	AES-256-GCM encryption of the plaintext using the CEK and IV.	Same length as the plaintext
5	Authentication tag	The AES-GCM tag.	Output of the encryption step. The recipient checks it to detect any change to parts 1, 3 or 4.	16 bytes (128 bits)

Protected header parameters

Decoded form of Part-1:

```
{
  "alg": "RSA-OAEP-256",
  "enc": "A256GCM",
  "kid": "aa-003-enc-2026-10",
  "cty": "JWT"
}
```

Part	Content	How it is produced	Size
alg	Yes	RSA-OAEP-256	How the CEK is wrapped (part 2).
enc	Yes	A256GCM	How the plaintext is encrypted (parts 3–5).
kid	Yes	Recipient's encryption key id as published in the CR	Tells the recipient which private key to use; enables key rotation.
cty	Yes	JWT	The plaintext is a signed message that must be verified after decryption.

Sequence of generation of Encrypted field (Initiator)

1. Build the API's business payload.
2. Sign it with the originator's private signing key as a compact JWS. The JWS header carries alg and the signer's kid.
3. Look up the recipient's current encryption public key (for RSA-OAEP-256) and the kid in the CR (the Target.id on a request; the original sender on a response).
4. Generate a fresh random 256-bit CEK and a fresh random 96-bit IV.
5. Build the protected header (alg, enc, kid, cty) and base64url-encode it. This is part 1.
6. Wrap the CEK with the recipient's public key using RSA-OAEP-256. This is part 2.
7. Encrypt the JWS with AES-256-GCM using the CEK, the IV and the ASCII of part 1 as additional authenticated data. This yields the ciphertext (part 4) and tag (part 5).
8. Join parts 1–5, each base64url-encoded without padding, with . to form iopPayload.
9. Discard the CEK. Build the envelope (ver, timestamp, txnid, correlationId, routing block (Target), iopPayload), sign it for x-jws-signature, and send the API key in the authorization header as applicable for the interface.

Sequence of reading of Encrypted field (Target)

1. Validate the hop: the caller's API key and the envelope's x-jws-signature against the caller's public key in the CR.
2. Split iopPayload on . and check that there are exactly five parts.
3. Decode the protected header; reject any alg or enc not on the allowed list.
4. Use kid to select the matching private key and unwrap the CEK with RSA-OAEP-256.
5. Decrypt with AES-256-GCM using the IV and the ASCII of part 1 as additional authenticated data. Reject if the tag check fails.
6. Verify the inner JWS against the signer's public key in the CR (selected by the JWS kid).
7. Check the signer: it must equal Initiator.id on a request, or the Target.id the request was sent to on a response. Reject otherwise.
8. Process the business payload.

Note

The same procedure applies, when an AA sends a request to FIP, or an FIP sends a notification to AA, or an AA sends a notification to FIU, which are not connected to them.